TECHHAVEN STUDIOS

HAVENDOS ROAD TO 1.0

The release plan from v0.8.2 to 1.0
Ten releases, grounded in the source tree

APPLICATIONS	##########################	45 apps, every one at 80x25
FILESYSTEM	##########################	FAT12 + FAT16 + FAT32
BLOCK LAYER	######..................	ATA, VirtIO, NVMe read
TRANSPORT	....	no BOT, no SCSI-over-USB
CONTROLLERS	######..................	UHCI only, no AHCI, no xHCI
PCI BUS	##########################	finds every function

DOCUMENT	THS-RM-1002 REV B · 12 OCT 2026
AUTHOR	Kyan Hoekstra / TechHaven Studios
BASELINE	v0.8.2 “Bedrock”, 1014 checks across 17 suites
SUPERSEDES	THS-RM-1001 sections 6, 7 and 13
STATUS	PROPOSAL — approved, not yet started

Contents

1 What changed since THS-RM-1001 3
2 Ground truth after 0.8.2 5
3 What 1.0 means 6
4 How the releases are written 7
5 The release map 8
6 0.9.0 “Ledger” — the additive installer 9
7 0.9.1 “Anchor” — NVMe write 11
8 0.9.2 “Pane” — any grid, chosen at boot 13
9 0.9.3 “Spindle” — every drive the machine has 15
10 0.9.4 “Digit” — xHCI and USB HID 17
11 0.9.5 “Courier” — USB mass storage 19
12 0.9.6 and 0.9.7 — the app port 20
13 0.9.8 “Signal” — a real NIC, and a browser 22
14 1.0 24
15 Cross-cutting work 25
16 Sequencing 27
17 Release gates 28
18 Deliberately not in 1.0 29
19 Open questions 30
20 How all of it is verified 31

1 What changed since THS-RM-1001

RM-1001 planned 1.0 as seven phases, A to G, around one sentence. Phase A shipped as v0.8.2 “Bedrock”. Four new requirements arrived after it, and three of them are not in RM-1001 at all:

1. Install on **every drive** — SATA, IDE, NVMe, USB, on any machine.
2. **USB 3.0 and up**, which means xHCI, which also means USB HID.
3. **Fill a 1080p screen**, picking the best resolution automatically at boot.
4. **Web browsing.**

This document is written from a second reading of the tree, after 0.8.2. Three findings decide the sequencing, and all three make the work larger than RM-1001 assumed.

FINDING 1

The console cannot express a 1080p grid. `vga_set_mode()` whitelists exactly four grids; `fbc_fit()` clamps to 90x60; and `shadow[]` and `fbc_painted[]` are statically sized for 5,400 cells against the ~32,400 a 1080p grid needs. Fill the screen is a driver change before it is an app change, and only then a port of ~550 hardcoded draw sites.

FINDING 2

There is no AHCI, no xHCI, no EHCI, no USB mass storage and no USB HID. A grep for `ahci_`, `xhci_` and `nvme_write` returns zero symbols. Worse and smaller: `ata.c` finds exactly one drive — it breaks out of the probe loop on the first hit — so no machine with two disks has ever been fully visible.

FINDING 3

TLS cannot reach the modern web and is not close. TLS 1.2, static RSA, the single suite 0x003C, no certificate validation, and `tls_crypto.c` holds no elliptic-curve maths, no GCM and no signature *verify* at all. Decision taken: HTTP-only browser in 1.0. Modern TLS is its own project and it is 1.1.

One correction to the machine, not the plan

RM-1001 and the handoff both assumed the HP's keyboard was USB behind xHCI and therefore dead. It is not: the embedded controller emulates the legacy 8042 and **the keyboard already works**. The **trackpad does not**. That takes xHCI off the critical path for basic usability and puts it on the pointer, which is a much less urgent place for it.

2 Ground truth after 0.8.2

What is actually in the tree, checked file by file. This table is the thing to argue with if any of it looks wrong.

Layer	In the tree	Not in the tree
PCI	Full function-level walk, follows bridges, 8 functions per device	—
Storage	ATA PIO (one drive), VirtIO, NVMe read, ATAPI on legacy ports	AHCI, NVMe write, multi-drive ATA, real LBA48
USB	UHCI controller, control + bulk, device descriptor only	xHCI, EHCI, configuration walk, BOT, SCSI, HID
Filesystem	FAT12, FAT16, FAT32 read+write+LFN, RAMFS	exFAT, NTFS, FAT16 long names
Partitioning	MBR read+write, GPT read	GPT write, ESP creation
Display	Framebuffer console, 4 fixed grids, centred, UEFI GOP	Any grid, auto-detect, apps that fill one
Input	PS/2 keyboard and mouse	USB HID, I2C-HID
Network	TCP/IP, DHCP, DNS, HTTP, TLS 1.2 static RSA, RTL8139, e1000	RTL8168, any 2006+ NIC, modern TLS, HTML
Audio	PC speaker beeps	HDA, AC'97, PCM, WAV, MP3
Boot	Hybrid BIOS + UEFI ISO, BIOS GRUB embedded	A UEFI loader it can install

> **NOTE**
> HavenDOS boots under UEFI already — that is how the HP runs it. What it cannot do is *install* a system that boots under UEFI, which needs an ESP and a PE32+ loader. Booting and installing are separate problems and only the first is solved.

3 What 1.0 means

RM-1001's sentence stands, with the four new requirements folded into it:

> **PROPOSED**
>
> HavenDOS 1.0 installs itself onto a computer made in the last twenty years — whatever the disk is attached to — boots from it under BIOS or UEFI, fills the screen it is given, reaches the network, and does so without damaging whatever was on the machine beforehand.

Six testable claims: **installs**, **boots**, **fills the screen**, **networks**, **takes input**, **harms nothing**. Section 17 turns each into a release gate.

"Every possible drive" is scoped honestly as IDE/PATA, SATA through AHCI, NVMe, VirtIO and USB mass storage. That is every consumer machine from roughly 2000 to now. SD/eMMC, SAS, hardware RAID and Intel VMD are out, and section 18 says why.

4 How the releases are written

Each release below has the same five parts, so they can be compared and picked up in whatever order the dependencies allow.

Part	What it answers
GOAL	What a person can do after it that they could not before.
WORK	Numbered items. Each is a sitting of work or close to it.
TEST	How it is proved. A suite that cannot fail proves nothing.
DONE	The acceptance check. Something observable, not “it compiles”.
RISK	What could go wrong and what to do about it.

Ten releases. Each ships a bootable ISO, a source archive and a changelog, as every milestone in this project has.

5 The release map

Release	Name	Unlocks
0.8.2	Bedrock	FAT32 — **done**
0.9.0	Ledger	Additive GPT + ESP + UEFI install, on images only
0.9.1	Anchor	NVMe write — **the HP installs and dual-boots**
0.9.2	Pane	Any grid, auto-detected — **the HP looks right**
0.9.3	Spindle	AHCI + multi-drive ATA — every SATA/IDE machine
0.9.4	Digit	xHCI + USB HID — pointer and external keyboards
0.9.5	Courier	USB mass storage — boot and install from a stick
0.9.6	Frame	App port wave 1 — the screens nobody can skip
0.9.7	Mosaic	App port wave 2 — the remaining ~35 apps
0.9.8	Signal	RTL8168 + the HTTP text browser
1.0	—	Release gates, manual, the carried-over bugs

The order is chosen so the HP works as early as possible without making every other machine an afterthought: it installs at 0.9.1, looks right at 0.9.2, and has a pointer at 0.9.4. Breadth for other people's machines lands at 0.9.3 and 0.9.5.

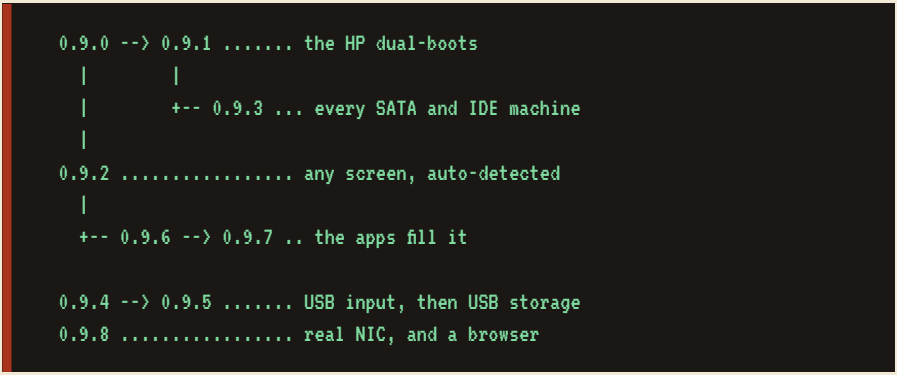


FIGURE 5-1 What blocks what. Only three edges are hard prerequisites: 0.9.1 cannot land before 0.9.0, 0.9.5 cannot land before 0.9.4, and the app port cannot start before 0.9.2.

6 0.9.0 “Ledger” — the additive installer

> **SIZE** Large
> **DEPENDS** Phase A — the ESP is FAT32
> **UNLOCKS** Dual boot. This is the phase that was actually asked for.
> **RISK** Highest in the document. It writes to a disk with Windows on it.

GOAL. HavenDOS adds itself to a disk that already has an operating system, rather than replacing the partition table.

> **CAUTION**
> Nothing in this release touches a real disk. Every write goes to a QEMU image file, where a bug costs a scratch file. Only once it is proved there does 0.9.1 enable a real write path.

WORK

1. **The byte-identity test, written first.** Build a Windows-shaped image — protective MBR, GPT, ESP, Microsoft Reserved, a large NTFS-sized partition — checksum every sector outside the region the installer may touch, run the install, and assert those sectors are identical afterwards. Written *before* the GPT writer, not after.
2. **GPT write, additive only.** Find free space between existing partitions, claim the first free slot in the entry array, leave every other entry untouched. Order matters: recompute the entry-array CRC first, then the header CRC over the header with its own CRC field zeroed. Write the primary at LBA 1 and the backup at the last LBA with its own copy of the entries. Never rewrite the table wholesale and never zero a header, not even briefly.

3. **ESP install.** Mount the EFI System Partition that is already there — the one Windows uses — through `fat32_scan_volumes()`, and create /EFI/HAVENDOS/ beside what is in it.
4. **A UEFI loader to install.** Both current embeds are `grub-mkimage -O i386-pc`, which is BIOS only. Add a third built `-O x86_64-efi` and write it to \EFI\BOOT\BOOTX64.EFI.
5. **Streaming self-hosting install.** An installed system still cannot install HavenDOS: the two-pass build means its kernel carries no embedded image. `fat32_read_at()` exists; `fat16.c` needs the same entry point and `write_file()` needs a variant that pulls a cluster at a time. The kernel is 631 KB and the heap is 255 KB, so it cannot be buffered.
6. **Invert the refusals.** Today GPT means refuse. After this it means: refuse when there is no free space, when the ESP is full, when the table is unreadable, when the disk is too small. The refusal machinery stays; only its reasons change.

> **NOTE**
> Creating a *second* ESP is legal and is the wrong answer. Firmware usually picks the first one it finds, so a second ESP produces an install that completes successfully and never boots — the worst failure shape there is, because nothing reports an error.

DONE

A QEMU disk with Windows-shaped partitioning boots HavenDOS from its own new partition under OVMF; every other partition is byte-identical by checksum; and a disk with no free space is refused with a message that says why.

7 0.9.1 “Anchor” — NVMe write

SIZE	Small in code, large in consequence
DEPENDS	0.9.0, proved
UNLOCKS	The HP dual-boots

GOAL. The owner’s laptop runs HavenDOS from its own internal disk, beside Windows.

WORK. `nvme_write()` is opcode 0x01 against the same I/O queue as 0x02 — perhaps forty lines against a driver that already works. Then wire NVMe into `inst_disk_write()`, remove the blanket read-only refusal, and make NVMe rows selectable in the drive picker with their free space shown.

> **CAUTION**
> Forty lines is exactly why this is sequenced after 0.9.0 and not beside it. A write path to a real NVMe disk with no additive installer behind it is a loaded gun in the tree: the only thing standing between a bug and a wiped Windows install would be the refusals, and those are meant to be the second line, not the first.

GATE BEFORE STARTING

Confirm the HP’s NVMe is visible at all. Many 11th and 12th generation Intel laptops ship with **RST/VMD enabled**, which hides the NVMe behind an Intel Volume Management Device where a standard NVMe driver sees nothing. If `disk` shows no NVMe on the HP, that is the reason, and the fix is a firmware setting — RST to AHCI/NVMe — not kernel work.

DONE

HavenDOS installed on the HP’s NVMe beside Windows, booting under UEFI from the internal disk with no removable media

attached, and Windows still boots. Every sector outside the new partition verified identical before and after.

8 0.9.2 “Pane” — any grid, chosen at boot

SIZE	Medium
DEPENDS	Nothing
UNLOCKS	The app port, and a machine that looks finished

GOAL. HavenDOS works out the best text grid for the screen it has been given, at boot, and uses it.

WORK

1. **Lift the grid cap.** Maximum columns 90 to 240 and rows 60 to 135. `shadow[]` and `fbc_painted[]` become 32,400 cells each — about 127 KB of additional BSS against a 4 MB kernel. Replace the four-mode whitelist with a validated arbitrary grid.
2. **Auto-detect.** A new `fbc_auto()` reads the real framebuffer and picks the largest grid whose glyphs stay legible, keeping the 2x-wide by 4x-tall proportion that gives HavenDOS its look.
3. **Ask the firmware for the panel’s own mode.** The Multiboot video tag currently requests 1280x800. Add `gfxpayload=keep` to the ISO’s grub.cfg so UEFI hands over the native mode, and let `fbc_auto()` work from what actually arrives.
4. **Fix the mouse clamp.** The pointer is pinned to 79x24 as literals. Until that changes nothing wider is clickable, including the start button.
5. **The viewport compatibility layer** — see section 15. Without it this release is a regression.
6. **Port the always-on chrome** in the same release: the desktop background, the bars, the corner clock, the cursor clamp, the taskbar and the shell.

Framebuffer	Grid today	Grid after 0.9.2
1280x800	80x25 centred	80x25, byte-identical
1920x1080	80x25 in a 1920x1080 margin	120x33

Framebuffer	Grid today	Grid after 0.9.2
1366x768	80x25 centred	85x24
1024x768 (OVMF)	80x25 centred	64x24
3840x2160	80x25 centred	120x33 at double the scale

> **NOTE**
> The chosen grid and *the numbers it was chosen from* go in the boot log through `klog()`. A screen that announces its resolution from a string literal is the thing `check_boot_log.py` exists to stop, and `neofetch` was caught doing exactly that in 0.8.2.

DONE

The HP boots to a desktop that fills its screen, and every app not yet ported appears exactly as it does today.

9 0.9.3 “Spindle” — every drive the machine has

SIZE	Large
DEPENDS	The MMIO seam — section 15
UNLOCKS	Every SATA machine from roughly 2008, and second disks

WORK

1. **drivers/ahci.c.** PCI class 01/06/01, ABAR is BAR5 and is MMIO. Per port: a 32-slot command list, a FIS receive area, command tables and a PRDT. READ DMA EXT (0x25) and WRITE DMA EXT (0x35) in a Host-to-Device Register FIS, polling PxCI the way the NVMe driver polls its phase tag. Enumerate every implemented port as a separate disk, not just port 0.
2. **ata.c: find all four drives.** Remove the break in the probe loop, keep a table instead of one active pointer, and give every entry point a drive index. Fix LBA48 while in there: it takes a 32-bit LBA and hardcodes the high bytes to zero, and selects LBA48 by capacity rather than by IDENTIFY word 83.
3. **ATAPI over AHCI.** The CD driver probes 0x1F0 and 0x170 only, so on any Q35/ICH9 machine the CD is invisible and `update` cannot read the ISO. The boot log has been saying so since 0.8 Beta.
4. **Route it all through the installer’s disk table** as new DTYPE_ values, and into the drive picker, which already knows how to label a type and refuse a row.

CHECK BEFORE STARTING

AHCI needs physically contiguous, aligned DMA memory — the command list wants 1 KB alignment and the FIS area 256 bytes. Confirm the physical memory manager can do aligned contiguous allocation before designing around it; if it cannot, that is a small prerequisite and far better found now than halfway through a driver.

DONE

A QEMU machine with an ich9-ahci controller and two disks lists both; a file written from inside HavenDOS is readable from the host image afterwards; then the same on real SATA hardware.

10 0.9.4 “Digit” — xHCI and USB HID

SIZE	Largest single driver in the plan
DEPENDS	The MMIO seam
UNLOCKS	Pointer input, external keyboards, and USB storage after it

GATE — ONE BOOT ANSWERS IT

Run `lspci -v` and `usb -vv` on the HP and record what the trackpad is. On most 12th-generation laptops the touchpad is **I2C-HID behind Intel’s Serial-IO controller, not USB**. If that is what it is, this release gets external USB mice and keyboards working and will not fix the trackpad — that needs an I2C-HID driver, which is separate work and probably not 1.0. Decide with the answer in hand, not before.

WORK

1. **drivers/xhci.c.** Capability registers, then operational registers at CAPLENGTH. A Device Context Base Address Array, scratchpad buffers, a command ring, an event ring with its segment table, a transfer ring per endpoint, and the doorbell array. The cycle bit is the thing to get right: every ring is a circular buffer whose producer toggles a bit on wrap, and a mismatch produces a controller that silently does nothing.
2. **drivers/usb_core.c.** Enumeration that is not tied to a controller: a device handle with a transport behind it, so UHCI and xHCI both satisfy it. The descriptor walk down device, configuration, interface and endpoint, then SET_ADDRESS and SET_CONFIGURATION. Today the UHCI probe stops at the device descriptor and never reads a configuration, so no endpoint has ever been discovered.
3. **drivers/usb_hid.c.** The HID *boot protocol*, deliberately, rather than full report-descriptor parsing: SET_PROTOCOL(0)

gives a fixed 8-byte keyboard report and a 3-byte mouse report. Feed them into the existing keyboard and mouse queues so nothing above the driver changes.

4. **Hub support.** Real machines put internal devices behind a root hub; without it they are invisible.

> **NOTE**
> This is the first interrupt-driven input path in a kernel that is polled everywhere on purpose, because IRQs are unreliable under UTM SE's interpreter. Polling HID at a low rate is viable and should be the default, with interrupts opt-in — and it needs saying out loud in the changelog, the way the audio exception was.

DONE

A USB mouse moves the pointer on real hardware, an external USB keyboard types, and `usb` lists devices behind an xHCI that has no UHCI companion.

11 0.9.5 “Courier” — USB mass storage

SIZE	Medium
DEPENDS	0.9.4 for xHCI; works on UHCI too
UNLOCKS	Machines with no SATA, and install media that is not a CD

WORK

1. **Bulk-Only Transport.** A 31-byte command block wrapper with signature 0x43425355, the data stage, then a 13-byte status wrapper with signature 0x53425355. Tags must match and residue must be honoured.
2. **SCSI.** INQUIRY, TEST UNIT READY, REQUEST SENSE, READ CAPACITY(10), READ(10), WRITE(10). Expect a unit-attention sense on first contact and retry rather than failing — a stick that has just been enumerated reports media change once.
3. **Register as a block device.** A DTYPE_USB in the installer’s disk table and a route into the FAT mount path. Without this the other two are a device list.

> **CAUTION**
> A stalled bulk endpoint needs CLEAR_FEATURE(ENDPOINT_HALT) **and** a toggle reset. The driver keeps the data toggle per address, endpoint and direction; if the two disagree, every later transfer is silently discarded by the device and nothing reports an error.

DONE

A USB stick with a known file on it, in QEMU and then in the HP: `usb` lists it as mass storage, the installer’s picker offers it, and the file reads back byte for byte.

12 0.9.6 and 0.9.7 — the app port

SIZE	Two releases, many small items
DEPENDS	0.9.2 and its viewport layer
UNLOCKS	A system that fills the screen rather than sitting in it

Roughly 550 literal row and column sites across the shell, concentrated in six files.

File	Suspicious sites	Lines
shell/desktop.c	213	4056
shell/shell.c	102	3807
shell/setup.c	74	395 — the densest in the tree
shell/packages.c	68	778
shell/iso_installer.c	57	590
shell/login.c	48	871

The pattern to follow already exists in the tree and should be copied rather than invented: named macros derived from `vga_rows()` and `vga_cols()` with the 80x25 value pinned in a comment beside each; clamp-both-ways accessors for viewports; and function-backed accessors where the value depends on content as well as screen size.

Four failure shapes, in increasing cost

	Shape	Cost
C	A fixed window origin, internally consistent — Tetris, Pong, the notification centre, the Install app	One or two constants each. Do these first.
D	Centring divided against a literal 80	Six sites, mechanical
A	Raw literal x and y in every draw call	The bulk of the work

	Shape	Cost
B	A row equality used as a hit test	112 sites, and these break clicking rather than painting

0.9.6 “Frame” takes the screens nobody can skip and which a new user meets first: the setup wizard, the ISO installer, login, Terms and Privacy, Settings, the dashboard and the file manager. **0.9.7 “Mosaic”** takes the remaining 35 registry apps, the games, the screensavers, the package apps and the lock screen.

TEST

Per app, the technique already used for the window chrome rather than the one used for the menus: a generator extracts the *real* draw function verbatim into a generated include, the test renders it into a fake oversized screen and asserts cell by cell where things land. A hand-copied duplicate drifts and starts certifying code that is no longer what ships.

> **NOTE**
> Two assertions on every ported app, both of which the existing geometry suites already make: **80x25 is byte-identical to before**, and **the old literal is no longer the right answer**. The second is what catches a regression back to a hardcoded row.

13 0.9.8 “Signal” — a real NIC, and a browser

SIZE	Medium
DEPENDS	Nothing
UNLOCKS	BOOT Chat on real hardware, and reading the web

WORK

1. **drivers/rtl8168.c.** Despite sharing a vendor id with the 8139 this is a different chip: PCIe, descriptor rings for transmit and receive rather than the 8139’s single circular buffer, and PHY setup over MDIO. Cover the 8168/8111/8411 family and the 8101/8102 fast-ethernet parts while in there; they are the same driver shape. If it presents the same interface as the e1000 driver, nothing above it changes and BOOT Chat starts working untouched.
2. **net/html.c.** A tokeniser, entity decoding, a block and inline model, a real word-wrapper and a link table. The manual renderer wraps by character despite its comment saying otherwise, so the wrapper is new code rather than a reuse.
3. **Parse Content-Type.** HTTP currently reads only Content-Length and Transfer-Encoding, so nothing can tell HTML from a binary and there is no charset. Widen the 256-byte path buffer while in there; it truncates real query strings silently.
4. **shell/browse.c.** Fetch, render, page, follow numbered links, history. The dmesg pager is the reusable part. Page memory comes from the physical allocator, not the heap: the heap is 256 KB with forward-only coalescing, while the PMM has around 252 MB free on a 256 MB machine.

> **CAUTION**
>
> **HTTPS does not work and this release does not change that.** TLS 1.2 with static RSA and one CBC suite is refused by essentially every modern server. `browse https://` must fail with a message that says exactly why, rather than hanging until a timeout and looking like a bug.

DONE

`ifconfig` shows a DHCP address on the real HP, BOOT Chat connects, and `browse` renders a real HTTP page with working links.

14 1.0

- The release gates in section 17, each on hardware rather than in QEMU.
- The standing ones: full suite green, help checker consistent, the build reproducible from the source archive in an empty directory.
- **The FAT16 dotfile bug.** `name_to_83()` turns `.desktop` into a blank basename with extension DES. Nine files are affected and it has been carried since before 0.8. The FAT32 driver's 8.3 check is the shape the fix should take: one guard, on an empty basename, with a long name written beside the mangled short one.
- `tests/test_fat16path.c` still skips. Give it a generated fixture or delete it; a suite that reports nothing is worse than no suite.
- The manual and its screenshots regenerated from a running system.

15 Cross-cutting work

The MMIO seam — before the first MMIO driver

`include/io.h` carries a real include guard as well as its pragma, specifically so a test can define it and supply its own port I/O. Every port-I/O driver reaches hardware through those six functions and nothing else, which is how PCI enumeration and the USB controller are tested without booting anything — 26 and 110 checks respectively.

> **PREREQUISITE**
>
> **MMIO has no such seam.** The NVMe and UHCI drivers dereference volatile pointers directly. AHCI, xHCI and any future HDA are all MMIO devices, so without a second seam all three are untestable except by booting. Propose `include/mmio.h` with the same guard trick, adopted before the first of the three rather than retrofitted into all of them — and adopted in the NVMe driver first, where a passing suite already exists to catch a mistake.

The viewport layer — so the app port can be incremental

The desktop already wraps every app launch in a screen clear, precisely so an 80x25 app does not leave desktop chrome around itself; the comment on it names the real fix. Add a push and pop of a viewport that centres a sub-grid inside the real one and offsets every positioned draw. An app not yet ported runs inside a centred 80x25 viewport and looks exactly as it does today; a ported app takes the full grid; one flag per entry in the app registry says which.

> **NOTE**
> This is what makes 0.9.6 and 0.9.7 safe to do app by app instead of as one 550-site commit. Without it, 0.9.2 is a regression: a 120x33 console with apps still drawing 80x25 puts them in the top-left corner, which is worse than the centred box they sit in today.

16 Sequencing

Only three edges are hard prerequisites. Everything else can move.

This	cannot land before	because
0.9.1 NVMe write	0.9.0 additive installer	a write path with no additive installer behind it is a loaded gun
0.9.5 USB storage	0.9.4 xHCI and enumeration	bulk transfers need endpoints, which need a configuration walk
0.9.6 app port	0.9.2 console and viewport	there is no larger grid to port to, and no way to not regress
0.9.3 AHCI, 0.9.4 xHCI	the MMIO seam	otherwise neither can be tested without booting

SEQUENCING RULE, CARRIED FROM RM-1001

NVMe write does not land before the additive installer is proved. It is perhaps forty lines against a driver that already works, which makes it tempting to do early. The only thing standing between a bug and a wiped Windows install would be the refusals, and those are meant to be the second line of defence, not the first.

17 Release gates

The six claims in section 3, each turned into something observable. 1.0 ships when all six pass on hardware, not in QEMU.

Claim	Gate
Installs	Installs onto a GPT disk with another OS on it, from the installer UI, with no hand-editing
Boots	The machine boots the installed system under UEFI with no removable media attached
Fills the screen	The desktop and every app occupy the whole panel on a 1080p machine, at a grid chosen without being told
Networks	`ifconfig` shows a DHCP address and BOOT Chat connects
Takes input	Keyboard and a pointing device both work on a machine with no PS/2 port
Harms nothing	Every sector outside the installed partition is byte-identical before and after, proved by checksum, and the other operating system still boots

18 Deliberately not in 1.0

Left out	Why
Modern TLS, and HTTPS	ECDHE or TLS 1.3, AES-GCM, signature verify and a CA store. None of the required maths exists. Larger than xHCI, and it is 1.1.
Wi-Fi	Signed firmware blobs per chipset and a full 802.11 MAC. Out of reach for a solo bare-metal kernel; the honest answer is Ethernet or nothing.
I2C-HID trackpads	Pending the gate in section 10. If the HP's trackpad is I2C it is a new driver, and a new release, not a line item.
Intel VMD / RST	A firmware setting solves it in seconds; a driver would take weeks and help only machines whose owner will not change that setting.
SD, eMMC, SAS, hardware RAID	Each is a controller family of its own. "Every drive" is scoped to IDE, AHCI, NVMe, VirtIO and USB.
NTFS	Reading a Windows partition is not needed in order to live beside one.
Secure Boot	Requires a signed loader. Users can turn it off, and 1.0 does not need to solve it.
Multitasking	Orthogonal to running on modern hardware, and large.
Audio	RM-1001's Phase F. Nothing in the four new requirements needs it, so it moves behind them.

19 Open questions

Four, and each is one boot on the HP. Each changes the plan, so they are worth answering before the release they affect rather than during it.

1. **What bus is the trackpad on?** USB means 0.9.4 fixes it. I2C-HID behind Intel Serial-IO means 0.9.4 does not, and an I2C-HID driver is a new item outside this plan.
2. **Is the HP's NVMe visible, or hidden behind RST/VMD?** If `disk` shows no NVMe, 0.9.1's acceptance cannot be met until the firmware setting is changed.
3. **Does the HP have an Ethernet socket at all?** Many 15-inch 2023 laptops do not. If not, 0.9.8 is still correct for desktops and other machines, but BOOT Chat will not work on that one and the honest answer there is Wi-Fi, which section 18 rules out.
4. **What does the firmware hand over with gfxpayload=keep?** Decides whether 0.9.2's auto-detect sees 1920x1080 or something the panel scales.

20 How all of it is verified

None of this is new. It is what the tree already enforces, applied to ten more releases.

- **make test green**, growing per release. 1014 checks across 17 suites today.
- **A fixture a suite needs is generated by the build**, or the suite is decoration. One suite sat in the output with no result for three releases because its fixture never existed and a `tail -1` ate the skip notice.
- **Every new driver gets an injection harness.** Inject real bugs and require the suite to notice each one. The FAT32 harness found ten holes in a suite that was reporting 168 of 168; the UHCI one found three. A suite that cannot fail proves nothing.
- **Byte-identity before and after** for anything that writes to a disk, proved by checksum, on an image first and hardware second.
- **Boot tests on real paths**, BIOS and under OVMF, not just native compiles.
- **No invented output.** The boot-log checker enforces it in the boot path. `neofetch` was found describing absent hardware in string literals outside that path and was rewritten in 0.8.2; any new status screen is held to the same rule.